

Chapter 8

6)

```
int sevenSegmentDigits[10][7] = { {1, 1, 1, 1, 1, 1, 0},  
                                     {0, 1, 1, 0, 0, 0, 0},  
                                     {1, 1, 0, 1, 1, 0, 1},  
                                     {1, 1, 1, 1, 0, 0, 1},  
                                     {0, 1, 1, 0, 0, 1, 1},  
                                     {1, 0, 1, 1, 0, 1, 1},  
                                     {1, 0, 1, 1, 1, 1, 1},  
                                     {1, 1, 1, 0, 0, 0, 0},  
                                     {1, 1, 1, 1, 1, 1, 1},  
                                     {1, 1, 1, 1, 0, 1, 1}};
```

10)

```
char chess_board[8][8] = { {'r', 'n', 'b', 'q', 'k', 'b', 'n', 'r'},  
                            {'p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'},  
                            {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},  
                            {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},  
                            {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},  
                            {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},  
                            {'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'},  
                            {'R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R'}};
```

11)

```
#include <stdio.h>
```

```
int main()
```

Homework 4 Nicholas Galus

```
{  
  
    char checkr_board[8][8];  
    for (int row = 0; row < 8; row++)  
    {  
        for (int col = 0; col < 8; col++)  
            (row + col) % 2 == 0 ? checkr_board[row][col] = 'B' :  
            checkr_board[row][col] = 'R';  
    }  
  
    for (int row = 0; row < 8; row++)  
    {  
        for (int col = 0; col < 8; col++)  
            printf("%c ", checkr_board[row][col]);  
  
        printf("\n");  
    }  
  
    return 0;  
}
```

Programming Project 9)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
int main(void) {
```

```
    int xPos = 0, yPos = 0;
```

Homework 4 Nicholas Galus

```
char walk[10][10] = {{',', ',', ',', ',', ',', ',', ',', ',', ',', ','},
    {',', ',', ',', ',', ',', ',', ',', ',', ',', ','},
    {',', ',', ',', ',', ',', ',', ',', ',', ',', ','},
    {',', ',', ',', ',', ',', ',', ',', ',', ',', ','},
    {',', ',', ',', ',', ',', ',', ',', ',', ',', ','},
    {',', ',', ',', ',', ',', ',', ',', ',', ',', ','},
    {',', ',', ',', ',', ',', ',', ',', ',', ',', ','},
    {',', ',', ',', ',', ',', ',', ',', ',', ',', ','},
    {',', ',', ',', ',', ',', ',', ',', ',', ',', ','}};
```

```
srand(time(NULL));
```

```
walk[xPos][yPos] = 'A';
```

```
for (int i = 'B'; i <= 'Z'; i++) {
```

```
    int validMove = 0;
```

```
    int attempts = 0;
```

```
    int newX, newY;
```

```
    while (!validMove && attempts < 100) {
```

```
        int direction = rand() % 4;
```

```
        newX = xPos;
```

```
        newY = yPos;
```

Homework 4 Nicholas Galus

```
switch (direction) {  
    case 0: // Move right  
        if (yPos < 10 - 1) newY += 1;  
        break;  
    case 1: // Move down  
        if (xPos < 10 - 1) newX += 1;  
        break;  
    case 2: // Move left  
        if (yPos > 0) newY -= 1;  
        break;  
    case 3: // Move up  
        if (xPos > 0) newX -= 1;  
        break;  
}  
  
if (walk[newX][newY] == '.') {  
    xPos = newX;  
    yPos = newY;  
    walk[xPos][yPos] = (char)i;  
    validMove = 1;  
}  
  
attempts++;  
}
```

Homework 4 Nicholas Galus

```
if (!validMove) {  
    //printf("No more valid moves! Exiting early at character: %c\n", i);  
    break;  
}  
}  
  
for (int row = 0; row < 10; row++) {  
    for (int col = 0; col < 10; col++) {  
        printf("%c ", walk[row][col]);  
    }  
    printf("\n");  
}  
  
return 0;  
}
```

Chapter 9

2)

```
int check(int x, int y, int n)  
{  
    if (x >= 0 && x < n && y >= 0 && y < n)  
        return 1  
    return 0;  
}
```

Homework 4 Nicholas Galus

9)

i = 1, j = 2

11)

float compute_GPA(char grades[], int n)

```
{  
    int total_points = 0;  
    for (int i = 0; i < n; i++)  
    {  
        switch (tolower(grades[i]))  
        {  
            case 'a': total_points += 4; break;  
            case 'b': total_points += 3; break;  
            case 'c': total_points += 2; break;  
            case 'd': total_points += 1; break;  
            case 'f': total_points += 0; break;  
            default: break;  
        }  
    }  
  
    n > 0 ? return (float)total_points / n : {return 0.0;  
}
```